

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e3b53aed-f4e\agent-a853e787738e2e5e9.jsonl`
- SHA-256: `8c8d6d79d4e23d13488e95c40ec1034d8f1fb0cb48a814b5ae7d0bcae13992bb`
- Source modified: `2026-06-18T02:02:46+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_e3b53aed-f4e`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T02:00:03.273Z)

Review a bug-fix PR in badminton-highlight-indexer. Work READ-ONLY in this git worktree (do NOT modify): C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/db-migration-fix

First run `git -C "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/db-migration-fix" diff origin/master..HEAD`` to see the exact change, then read the touched files + related code/tests. PR #212 makes the SQLite v3/v4 migration ALTERs idempotent (backend/infrastructure/database.py)

so an interrupted upgrade (ALTER auto-commits, version bump doesn't) can't permanently brick the DB

with 'duplicate column name' on re-open. A new `_add_column_idempotent` helper swallows ONLY that `OperationalError`. Regression test `test_migration_idempotent_on_interrupted_upgrade` in `tests/test_database.py`.

Be adversarial ? try to find a real reason NOT to merge. Concrete, verifiable findings only (cite file:line); no style noise. severity must-fix only for a genuine bug/regression/break. `blocks_merge=true` ONLY on a real must-fix. Return the structured object.

LENS ? correctness: is the fix logically right? For #212: does swallowing ONLY 'duplicate column name' avoid masking other errors, and does the happy path still bump versions exactly as before? For #213: does `unwrap_or_failure` handle Success/Failure/bare-list correctly, and is the Failure branch (continue vs return) right for each script?

2. assistant (2026-06-18T02:00:05.934Z)

I'll review PR #212. Let me start by examining the exact change.

3. user (2026-06-18T02:00:07.191Z)

```
diff --git a/backend/infrastructure/database.py b/backend/infrastructure/database.py
index 59473ac..58153a9 100644
--- a/backend/infrastructure/database.py
+++ b/backend/infrastructure/database.py
@@ -41,6 +41,24 @@ class Database:
     pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
     return conn

+    @staticmethod
+    def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
+        """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.
+
+        A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn`` transaction, so an interrupted v3/v4 block can leave the column added while ``user_version`` stays un-bumped ? and the next open would re-run the same ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every future ``Database(...)`` construction (``_init_db`` runs in ``__init__``). Swallow
```

```

+         ONLY that specific error so the ladder stays crash-safe, matching the re-runnable
+         ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
+         """
+         try:
+             cursor.execute(ddl)
+         except sqlite3.OperationalError as exc:
+             if "duplicate column name" not in str(exc).lower():
+                 raise
+
+     def _init_db(self):
+         """Initializes tables for videos, play_segments, and events."""
+         with self._connect() as conn:
@@ -152,9 +170,10 @@ class Database:
+             # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
due
+             # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any
worker
+             # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
-             # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
-             cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
-             cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
+             # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
+             # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
them.
+             self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
+             self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
+             cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
+             cursor.execute("PRAGMA user_version = 3")

@@ -163,10 +182,11 @@ class Database:
+             # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
+             # path can scope rows to an owner. **NULL = local install = byte-identical to
+             # today** ? every existing row stays NULL and every existing query (which never
-             # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive.
+             # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive;
+             # the ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
them.
+             # Additive ONLY: no served-behavior change rides on this slice.
-             cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
-             cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
+             self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
TEXT")
+             self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
user_id TEXT")
+             # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path
free
+             # of index churn while making the eventual per-user lookups cheap.
+             cursor.execute(
diff --git a/tests/test_database.py b/tests/test_database.py
index 5ccdb49..8909208 100644
--- a/tests/test_database.py
+++ b/tests/test_database.py
@@ -244,6 +244,39 @@ def test_migration_v3_db_upgrades_to_v5_preserving_rows(tmp_path):
+     assert len(cands) == 1 and cands[0]["stats"]["net_crossings"] == 3

+def test_migration_idempotent_on_interrupted_upgrade(tmp_path):
+     """An INTERRUPTED v3/v4 upgrade must not brick the DB. A DDL ``ALTER`` auto-commits
+     on its own, so a crash after the ADD COLUMNS but before the ``PRAGMA user_version``
+     bump leaves the columns present while the version is stale ? re-opening re-runs the
+     bare ALTERs. The ladder must tolerate that (re-runnable, like the v1/v2/v5 blocks)

```

```

+ instead of raising ``OperationalError: duplicate column name``. """
+ db_path = tmp_path / "interrupted.db"
+ Database(str(db_path)) # fully migrate to the current version (v3/v4 columns all present)
+
+ # Simulate the interruption: the v3/v4 ADD COLUMNS already landed and auto-committed,
+ # but the process died before the version bump, so user_version is stuck at 2.
+ conn = sqlite3.connect(str(db_path))
+ conn.execute("DROP TABLE IF EXISTS user_models") # a true v2 DB lacks the v5 table
+ conn.execute("PRAGMA user_version = 2")
+ conn.commit()
+ conn.close()
+
+ # Re-opening re-runs v3 (jobs.policy/next_poll_at) and v4 (videos/candidate_segments
+ # .user_id) over columns that ALREADY exist ? must NOT raise, and must reach v5.
+ Database(str(db_path))
+ conn = sqlite3.connect(str(db_path))
+ assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
+ conn.close()
+ # Columns/tables are all present after the clean re-apply.
+ assert "policy" in _table_cols(db_path, "jobs")
+ assert "user_id" in _table_cols(db_path, "videos")
+ assert "user_id" in _table_cols(db_path, "candidate_segments")
+ conn = sqlite3.connect(str(db_path))
+ assert "user_models" in {r[0] for r in conn.execute(
+     "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
+ conn.close()
+
+
+ # --- v5: per-user model store CRUD -----
+
+ def test_user_models_schema_contract(tmp_path):

```

4. assistant (2026-06-18T02:00:10.550Z)

The diff touches PR #212 (database migration). The task description also mentions PR #213 (`unwrap\_or\_failure` in scripts) in the lens, but the diff only shows #212. Let me check whether #213 changes are also present and read the full migration code.

5. user (2026-06-18T02:00:11.952Z)

```

1     import sqlite3
2     import json
3     import logging
4     from contextlib import contextmanager
5     from typing import List, Dict, Any, Optional, Tuple
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         @contextmanager
15         def _connect(self):
16             """Transaction scope that also CLOSES the connection.
17
18             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19             never closes, so every call leaked its connection to GC (lingering file
20             handles on the WAL sidecars under Windows). Internal methods use this.
21             """
22             conn = self._get_connection()
23             try:
24                 with conn:
25                     yield conn

```

```

26         finally:
27             conn.close()
28
29     def _get_connection(self):
30         # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31         # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
32         conn = sqlite3.connect(self.db_path, timeout=30.0)
33         conn.row_factory = sqlite3.Row
34         # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
35         conn.execute("PRAGMA foreign_keys = ON")
36         # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
37         conn.execute("PRAGMA busy_timeout = 30000")
38         try:
39             conn.execute("PRAGMA journal_mode = WAL")
40         except sqlite3.OperationalError:
41             pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
42         return conn
43
44     @staticmethod
45     def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
46         """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runably.
47
48         A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn``
49         transaction, so an interrupted v3/v4 block can leave the column added while
50         ``user_version`` stays un-bumped ? and the next open would re-run the same
51         ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
52         future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
Swallow
53         ONLY that specific error so the ladder stays crash-safe, matching the re-
runnable
54         ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
55         """
56         try:
57             cursor.execute(ddl)
58         except sqlite3.OperationalError as exc:
59             if "duplicate column name" not in str(exc).lower():
60                 raise
61
62     def _init_db(self):
63         """Initializes tables for videos, play_segments, and events."""
64         with self._connect() as conn:
65             cursor = conn.cursor()
66
67             # Videos Table
68             cursor.execute("""
69                 CREATE TABLE IF NOT EXISTS videos (
70                     id TEXT PRIMARY KEY,
71                     filepath TEXT NOT NULL,
72                     processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
73                     status TEXT NOT NULL,
74                     validation_results TEXT
75                 )
76             """)
77
78             # Play Segments Table (Extensible metadata JSON)
79             cursor.execute("""
80                 CREATE TABLE IF NOT EXISTS play_segments (
81                     id INTEGER PRIMARY KEY AUTOINCREMENT,
82                     video_id TEXT NOT NULL,
83                     start_time REAL NOT NULL,
84                     end_time REAL NOT NULL,
85                     duration REAL NOT NULL,
86                     server TEXT,
87                     receiver TEXT,

```

```

88         metadata TEXT,
89         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
90     )
91     """)
92
93 # Events Table
94 cursor.execute("""
95     CREATE TABLE IF NOT EXISTS events (
96         id INTEGER PRIMARY KEY AUTOINCREMENT,
97         segment_id INTEGER NOT NULL,
98         timestamp REAL NOT NULL,
99         type TEXT NOT NULL,
100        player TEXT,
101        details TEXT,
102        FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
103    )
104    """)
105
106 # Versioned migrations live here. PRAGMA user_version is the schema
107 # contract ? bump it for every change and add an ordered `if version < N`
108 # block below. Tests assert the expected version, so accidental drift fails
CI.
109 version = cursor.execute("PRAGMA user_version").fetchone()[0]
110
111 if version < 1:
112     # v1: raw candidate detections (pre-confirmation), detector-agnostic.
113     # Common fields are columns; detector-specific metrics go in `stats`
JSON,
114     # so a new segmenter reuses this table by setting its own
`detector`/`stats`.
115     cursor.execute("""
116         CREATE TABLE IF NOT EXISTS candidate_segments (
117             id INTEGER PRIMARY KEY AUTOINCREMENT,
118             video_id TEXT NOT NULL,
119             detector TEXT NOT NULL,
120             chunk_index INTEGER,
121             start_time REAL NOT NULL,
122             end_time REAL NOT NULL,
123             duration REAL NOT NULL,
124             confidence REAL,
125             stats TEXT,
126             detection_params TEXT,
127             confirmed_segment_id INTEGER,
128             human_label TEXT,
129             notes TEXT,
130             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
131             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
132             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
133         )
134         """)
135     cursor.execute("""
136         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
137             ON candidate_segments(video_id, detector)
138         """)
139     cursor.execute("PRAGMA user_version = 1")
140
141 if version < 2:
142     # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
143     # /api/process request. `status` is the EXECUTION state of the job
144     # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
145     # that failed validation) lives inside `result` JSON as
result["status"].

```

```

146         # `result` is the full sync-era response dict (incl. report paths), so
the
147         # observability report is the job's artifact, per the plan.
148         cursor.execute("""
149             CREATE TABLE IF NOT EXISTS jobs (
150                 id TEXT PRIMARY KEY,
151                 video_path TEXT NOT NULL,
152                 video_id TEXT,
153                 segmenter TEXT,
154                 player_pool TEXT,
155                 max_frames INTEGER,
156                 status TEXT NOT NULL DEFAULT 'queued',
157                 progress TEXT,
158                 error TEXT,
159                 result TEXT,
160                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
161                 started_at TIMESTAMP,
162                 finished_at TIMESTAMP
163             )
164         """)
165         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
166         cursor.execute("PRAGMA user_version = 2")
167
168     if version < 3:
169         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
170         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
171         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
172         # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
173         # the coordinator. ALTER (not recreate) so existing v2 job rows survive;
the
174         # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-
run them.
175         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy
TEXT")
176         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN
next_poll_at TIMESTAMP")
177         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
178         cursor.execute("PRAGMA user_version = 3")
179
180     if version < 4:
181         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
NULLABLE
182         # `user_id` to videos + candidate_segments so a future per-user / multi-
tenant
183         # path can scope rows to an owner. **NULL = local install = byte-
identical to
184         # today** ? every existing row stays NULL and every existing query
(which never
185         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive;
186         # the ADD COLUMNS are applied idempotently so an interrupted upgrade can
re-run them.
187         # Additive ONLY: no served-behavior change rides on this slice.
188         self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN
user_id TEXT")
189         self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD
COLUMN user_id TEXT")
190         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
path free

```

```

191         # of index churn while making the eventual per-user lookups cheap.
192         cursor.execute(
193             "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
194             "WHERE user_id IS NOT NULL")
195         cursor.execute(
196             "CREATE INDEX IF NOT EXISTS idx_candidates_user "
197             "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
198         cursor.execute("PRAGMA user_version = 4")
199
200     if version < 5:
201         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per
202         (user_id,
203         # version): the resolved per-user `fusion_config` overlay + an INLINE
204         # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
205         evidence
206         # and provenance that earned it. `is_active` pins the one row the
207         serving bridge
208         # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`).
209         This slice
210         # only persists/loads it ? NOTHING reads it on the served path yet.
211         #
212         #   user_id           ? owner of the model (NULL allowed = the local
213         install)
214         #   version           ? monotonic per-user model version
215         #   baseline_version   ? the shared baseline this was fit against
216         (upgrade switch)
217         #   feature_schema_hash ? fingerprint of the feature set; MINOR-vs-
218         MAJOR re-fit gate
219         #   fusion_config      ? JSON per-user FusionConfig overlay (cue_flags
220         / thresholds)
221         #   classifier_json    ? JSON LogisticRegression.to_dict() (None =
222         config-only model)
223         #   promotion_evidence ? JSON per_user_gate PromotionDecision (why it
224         was promoted)
225         #   n_videos_at_fit    ? held-out-user corpus size when fit (min_n
226         trust floor input)
227         #   is_active          ? the one resolved row for this user (partial-
228         unique below)
229         cursor.execute("""
230             CREATE TABLE IF NOT EXISTS user_models (
231                 id INTEGER PRIMARY KEY AUTOINCREMENT,
232                 user_id TEXT,

```

6. user (2026-06-18T02:00:12.890Z)

d140722 fix(db): make v3/v4 migration ALTERs idempotent so an interrupted upgrade can't brick the DB

7. assistant (2026-06-18T02:00:18.397Z)

This PR only contains the #212 database change (one commit). The #213 lens in the task description doesn't apply here since there's no `unwrap\_or\_failure` change in this diff. Let me focus on #212 and verify the correctness carefully.

Let me read the test file context and verify the happy-path version bumping behavior, plus check the transaction semantics claim.

8. user (2026-06-18T02:00:20.403Z)

```

1     import sqlite3
2     from backend.infrastructure.database import Database
3
4     def test_database_init(tmp_path):
5         db_path = tmp_path / "test.db"

```

```

6         Database(str(db_path)) # instantiation creates the schema
7
8         conn = sqlite3.connect(str(db_path))
9         cursor = conn.cursor()
10        cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
11        tables = [r[0] for r in cursor.fetchall()]
12        assert "videos" in tables
13        assert "play_segments" in tables
14        conn.close()
15
16    def test_database_video(tmp_path):
17        db = Database(str(tmp_path / "test.db"))
18
19        vid_added = db.add_video("vid1", "vid1.mp4", "pending", [{"val": 1}])
20        assert vid_added is True
21
22        video = db.get_video("vid1")
23        assert video["id"] == "vid1"
24        assert video["filepath"] == "vid1.mp4"
25        assert video["status"] == "pending"
26        assert video["validation_results"] == [{"val": 1}]
27
28        db.clear_video_data("vid1")
29        assert db.get_video("vid1") is None
30
31    def test_database_play_segment(tmp_path):
32        db = Database(str(tmp_path / "test.db"))
33        db.add_video("vid1", "vid1.mp4", "pending", [])
34
35        sid = db.add_play_segment("vid1", 10.0, 20.0, "PlayerA", "PlayerB", {"foo": "bar"})
36        assert sid is not None
37
38        db.add_event(sid, 15.0, "smash", "PlayerA", {"speed": 300})
39
40        segments = db.query_play_segments("vid1", {"server": "PlayerA"})
41        assert len(segments) == 1
42        assert segments[0]["id"] == sid
43        assert segments[0]["start_time"] == 10.0
44        assert segments[0]["server"] == "PlayerA"
45        assert segments[0]["metadata"]["foo"] == "bar"
46
47        assert len(segments[0]["events"]) == 1
48        assert segments[0]["events"][0]["type"] == "smash"
49        assert segments[0]["events"][0]["details"]["speed"] == 300
50
51
52    # --- candidate_segments schema + behavior -----
53
54    # The schema contract: any future change to candidate_segments columns must
55    # consciously update this set, otherwise the test below fails (regression guard).
56    EXPECTED_CANDIDATE_COLUMNS = {
57        "id", "video_id", "detector", "chunk_index", "start_time", "end_time",
58        "duration", "confidence", "stats", "detection_params",
59        "confirmed_segment_id", "human_label", "notes", "created_at",
60        "user_id", # v4: personalization groundwork (nullable; NULL = local install)

```

9. user (2026-06-18T02:00:20.747Z)

```

220        user_id TEXT,
221        version INTEGER NOT NULL DEFAULT 1,
222        baseline_version TEXT,
223        feature_schema_hash TEXT,
224        fusion_config TEXT,
225        classifier_json TEXT,
226        promotion_evidence TEXT,

```



```

227             n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
228             is_active INTEGER NOT NULL DEFAULT 0,
229             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
230         )
231     """)
232     # One model version per user (NULLs compare distinct in SQLite, so the
local
233     # install can still carry multiple versioned rows; the active-row guard
below
234     # is what enforces a single served model).
235     cursor.execute("""
236         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
237         ON user_models(user_id, version)
238     """)
239     # At most ONE active model per user: a partial-unique index over the
active rows.
240     # (SQLite treats NULL user_id rows as distinct, so the local install's
single
241     # active row is still guarded ? there is one local user.)
242     cursor.execute("""
243         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
244         ON user_models(user_id) WHERE is_active = 1
245     """)
246     cursor.execute("PRAGMA user_version = 5")
247     # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6
248
249     conn.commit()
250
251     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
252         """Register or update a video row WITHOUT touching its child segments.
253
254         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
255         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
256         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
257         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
258         row in place; clearing segments is now an explicit, deliberate operation
259         (clear_video_data / prune_segments).
260         """
261         try:
262             with self._connect() as conn:
263                 conn.cursor().execute(
264                     "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
265                     "ON CONFLICT(id) DO UPDATE SET "
266                     "filepath=excluded.filepath, status=excluded.status, "
267                     "validation_results=excluded.validation_results",
268                     (video_id, filepath, status, json.dumps(validation_results))
269                 )
270                 conn.commit()
271             return True
272         except Exception as e:
273             logger.error(f"Failed to add video: {e}")
274             return False
275
276     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
277         """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
278
279         Captured before a (re)segmentation so the run's new rows (id > watermark) can be

```

10. assistant (2026-06-18T02:00:23.057Z)

Now let me verify the test helper `\_table\_cols` exists and run the test suite to confirm the fix actually works and doesn't break anything.

11. user (2026-06-18T02:00:25.041Z)

```
171: def _table_cols(db_path, table):
172-     conn = sqlite3.connect(str(db_path))
173-     cols = [c[1] for c in conn.execute(f"PRAGMA table_info({table})").fetchall()]
174-     conn.close()
175-     return cols
176-
177-
```

12. user (2026-06-18T02:00:27.577Z)

```
..... [100%]
20 passed in 0.83s
```

13. assistant (2026-06-18T02:00:33.379Z)

All 20 tests pass. Now let me adversarially probe a subtle correctness concern: the `version` variable is read **once** at the top (line 109) and all `if version < N` blocks key off that single snapshot. In the interrupted scenario, the test sets `user\_version=2`, so on re-open all blocks v3, v4, v5 run. That's fine.

But let me think harder about a real edge case the fix introduces. The key question: does swallowing "duplicate column name" mask a genuine partial-interruption bug where the SECOND ALTER in a block never ran? Let me trace the interrupted scenario precisely.

14. user (2026-06-18T02:00:42.034Z)

```
jobs cols: ['id', 'next_poll_at', 'policy', 'status']
version: 5
PARTIAL-BLOCK RECOVERY OK
```

15. assistant (2026-06-18T02:00:45.836Z)

Partial-block recovery works correctly ? the swallow handles the first ALTER, the second ALTER still runs. Now let me verify the two adversarial concerns about the error-matching string:

1. Does the substring match `"duplicate column name"` reliably match SQLite's actual error message across versions?
2. Could it accidentally swallow a legitimately different error?

16. user (2026-06-18T02:00:49.676Z)

```
sqlite version: 3.50.4
duplicate-col msg: 'duplicate column name: b'
no-such-table msg: 'no such table: nope'
would-be-swallowed? False
```

17. assistant (2026-06-18T02:00:56.672Z)

The error matching is correct:

- The duplicate-column error is `"duplicate column name: b"` ? the substring `"duplicate column name"` matches.
 - A genuinely different error (`"no such table: nope"`) does NOT match and would be re-raised.
- Good ? it doesn't mask the case where the `jobs` table was itself never created.

Now let me verify the happy-path version-bump behavior is byte-identical to before. On a fresh DB, none of the ALTERs raise, so `\_add\_column\_idempotent` just runs `cursor.execute(ddl)` and the version bumps proceed exactly as before. The earlier 20 passing tests (including

`test\_migration\_v3\_db\_upgrades\_to\_v5\_preserving\_rows` and the version-contract tests) confirm this.

Let me do one final adversarial check ? confirm there's no concern about the `version` snapshot causing a fresh-DB `version < 3` block to fail if `jobs` doesn't exist. On a fresh DB, `version` starts at 0, so v2 creates `jobs` before v3 ALTERs it. But what if someone has a v2 DB where `jobs` exists ? fine. Let me also confirm the full test suite collects cleanly for this module and check whether ruff/mypy are part of CI for this file.

18. user (2026-06-18T02:00:58.922Z)

All checks passed!

---MYPY---

Success: no issues found in 1 source file

19. assistant (2026-06-18T02:01:01.312Z)

Lint and type checks are clean. Let me do a final broad sanity check ? run the full test suite to confirm nothing else regressed (the migration helper touches a widely-used path).

20. user (2026-06-18T02:02:21.473Z)

```
=====
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
obsreport (optional reporting dep): available
=====
.....SS..... [ 8%]
..... [ 16%]
..... [ 25%]
..... [ 33%]
..... [ 42%]
..... [ 50%]
..... [ 59%]
..... [ 67%]
.....SS [ 75%]
SSS..... [ 84%]
..... [ 92%]
..... [100%]
847 passed, 7 skipped in 75.40s (0:01:15)
=====
RESULT: PASS - all tests green
=====
```

21. assistant (2026-06-18T02:02:33.963Z)

Full suite green: 847 passed, 7 skipped. Let me consolidate the review.

****Correctness analysis of PR #212:****

1. ****Swallow scope is correctly narrow.**** `\_add\_column\_idempotent` (database.py:44-60) catches `sqlite3.OperationalError` but re-raises unless `"duplicate column name"` is in the lowercased message. I verified empirically that SQLite 3.50.4 emits `"duplicate column name: b"` (matches) while a genuinely different failure like `"no such table: nope"` does NOT match and is re-raised. So an ALTER against a missing/broken `jobs` table ? a real corruption signal ? still surfaces. It does not mask other errors.

2. ****Happy path unchanged.**** On a fresh/normal upgrade the ALTERs don't raise, so the helper is a transparent pass-through to `cursor.execute(ddl)`. Version bumps (`PRAGMA user\_version = 3/4/5`) are untouched and run exactly as before. Confirmed by the unchanged version-contract tests passing.

3. ****The fix actually addresses the stated bug + a stronger case.**** I hand-simulated both (a) the documented interruption (columns landed, version stuck at 2) and (b) a harder partial-block

interruption (first ALTER landed, second never ran). Both recover cleanly and reach v5 with all columns present. The single `version` snapshot at line 109 means all `version < N` blocks re-run from the stale version, which is the correct behavior here.

4. **Note (not a defect):** The block as a whole still isn't atomic ? DDL ALTERs auto-commit, so an interruption mid-block always leaves a stale `user\_version`. This PR makes that re-runnable rather than fully transactional, which is the right and minimal fix and matches the existing `CREATE ... IF NOT EXISTS` pattern for v1/v2/v5. The CREATE INDEX statements were already idempotent.

The #213 lens in the task does not apply ? this branch contains only the single #212 commit; there is no `unwrap\_or\_failure` / scripts change in the diff.

No must-fix, should-fix, or nit findings. The fix is logically correct, minimal, well-tested, and lint/type-clean.

22. user (2026-06-18T02:02:40.200Z)

Structured output provided successfully

23. assistant (2026-06-18T02:02:46.064Z)

Review complete. PR #212 is correct and safe to merge ? no must-fix, should-fix, or nit findings.

Summary of what I verified:

- **Narrow swallow:** `\_add\_column\_idempotent` (database.py:44-60) only suppresses `OperationalError` matching `"duplicate column name"`; I empirically confirmed SQLite's actual message matches, and that an unrelated error like `"no such table"` is re-raised, so genuine corruption isn't masked.
- **Happy path unchanged:** the helper is a transparent pass-through when ALTERs don't raise; version bumps proceed exactly as before.
- **Recovers the bug + a harder case:** both the documented interruption and a partial-block interruption (first ALTER landed, second didn't) recover cleanly to v5 with all columns present.
- **Green:** full suite 847 passed / 7 skipped; ruff and mypy clean on both touched files.

Note: the #213 `unwrap\_or\_failure` lens in the task prompt doesn't apply ? this branch is a single commit (d140722) touching only `backend/infrastructure/database.py` and `tests/test\_database.py`; there's no scripts change in the diff.